

Transaction:

```
# TRANSACTION: Transferring a player
@app.route("/api/players/transfer", methods=["POST"])
def transfer_player():
    data = request.get_json()
    player_name = data["playerName"]
    new_team_name = data["newTeamName"]

    conn = get_connection()
    try:
        with conn.cursor() as cursor:
            cursor.execute("SET TRANSACTION ISOLATION LEVEL SERIALIZABLE") # only one
person can edit the document at a time, the strictest isolation level
        conn.begin()
        with conn.cursor() as cursor: # BEGIN
TRANSACTION; -> start the transaction
            # Find player by name
            cursor.execute("""
                SELECT playerId, firstName, lastName FROM Player
                WHERE firstName LIKE %s OR lastName LIKE %s
                LIMIT 1
            """, (f"%{player_name}%", f"%{player_name}%"))
            player = cursor.fetchone()
            if not player:
                conn.rollback()
                return jsonify({"error": "Player not found"}), 404

            # Find team by name
            cursor.execute("""
                SELECT teamId, name FROM Team
                WHERE name LIKE %s
                LIMIT 1
            """, (f"%{new_team_name}%",))
            team = cursor.fetchone()
            if not team:
                conn.rollback()
                return jsonify({"error": "Team not found"}), 404

            player_id = player["playerId"]
            new_team_id = team["teamId"]
```

```

# Get player's current team
cursor.execute("""
    SELECT
        t.teamId,
        t.name AS teamName,
        COUNT(*) AS gamesPlayed
    FROM PlayerGameStats pgs
    JOIN Team t ON pgs.teamId = t.teamId
    WHERE pgs.playerId = %s
    GROUP BY t.teamId, t.name
    ORDER BY gamesPlayed DESC
    LIMIT 1
""", (player_id,))
current = cursor.fetchone() # returns a single row

if not current:
    conn.rollback()
    return jsonify({"error": "Player has no game stats"}), 404

# Get the 5 games that will be moved (before updating)
# date is a reserved word in MySQL. Wrap it in backticks:
cursor.execute("""
    SELECT
        pgs.gameId,
        g.gameDate,
        pgs.points,
        pgs.rebounds,
        pgs.assists
    FROM PlayerGameStats pgs
    JOIN Game g ON pgs.gameId = g.gameId
    WHERE pgs.playerId = %s
    ORDER BY g.gameDate DESC
    LIMIT 5
""", (player_id,))
moved_games = cursor.fetchall()

# Update their 5 most recent games
cursor.execute("""
game information
    UPDATE PlayerGameStats
    SET teamId = %s
    # UPDATE; -> updating the

```

```

        WHERE playerId = %s
        AND gameId IN (
            SELECT gameId FROM (
                SELECT gameId
                FROM PlayerGameStats
                WHERE playerId = %s
                ORDER BY gameId DESC
                LIMIT 5
            ) AS recent
        )
        "", (new_team_id, player_id, player_id))

conn.commit() # COMMIT; -> save the transaction
return jsonify({
    "message": "Player transferred successfully",
    "playerName": f"{player['firstName']} {player['lastName']}",
    "fromTeam": current["teamName"],
    "toTeam": team["name"],
    "movedGames": moved_games
})
except Exception as e:
    conn.rollback() # ROLLBACK; -> undo the transaction
    return jsonify({"error": str(e)}), 500
finally:
    conn.close()

```

Trigger:

```
CREATE TRIGGER check_playerStats
BEFORE INSERT ON PlayerGameStats
FOR EACH ROW
BEGIN
    # Points can't be negative
    IF NEW.points < 0 THEN
        SIGNAL SQLSTATE '45000'           # user-defined exception
        SET MESSAGE_TEXT = 'Points cannot be negative'; # message for the exception
    END IF;

    # Minutes must be between 0 and 73 [48 + 25]
    IF NEW.minutes < 0 OR NEW.minutes > 73 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Minutes must be between 0 and 73';
    END IF;

    # if fg_percent (field goal percentage) is null, default to 0
    IF NEW.fg_percent IS NULL THEN
        SET NEW.fg_percent = 0.0;
    END IF;
END
```

Stored Procedure:

```
CREATE PROCEDURE GetPlayerPerformance(IN p_playerId INT)
BEGIN
    DECLARE playerExists INT;

    # Check if player exists before running queries
    SELECT COUNT(*) INTO playerExists
    FROM Player
    WHERE playerId = p_playerId;

    IF playerExists = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'ERROR! Player not found';
    ELSE
        # Player career stats per team
        SELECT p.firstName, p.lastName, t.name AS teamName,
            COUNT(*) AS gamesPlayed,
            ROUND(AVG(pgs.points), 2) AS avgPoints,
            ROUND(AVG(pgs.rebounds), 2) AS avgRebounds,
            ROUND(AVG(pgs.assists), 2) AS avgAssists,
            MAX(pgs.points) AS careerMaxPointsPerGame
        FROM PlayerGameStats pgs
        JOIN Player p ON pgs.playerId = p.playerId
        JOIN Team t ON pgs.teamId = t.teamId
        WHERE p.playerId = p_playerId
        GROUP BY p.firstName, p.lastName, t.name;

        # Player vs teammates comparison
        SELECT t.name AS teamName,
            (SELECT ROUND(AVG(points), 2)
             FROM PlayerGameStats
             WHERE teamId = t.teamId) AS teamAvgPoints,
            ROUND(AVG(pgs_player.points), 2) AS playerAvgPoints
        FROM PlayerGameStats pgs_player
```

```
    JOIN PlayerGameStats pgs2 ON pgs_player.teamId = pgs2.teamId
    JOIN Team t              ON pgs_player.teamId = t.teamId
    WHERE pgs_player.playerId = p_playerId
    GROUP BY t.name;
END IF;
END
```